Nombre: Jesús López Pérez
Email: jlopez6646@alumno.uned.es

Sistema de Gestión de la Movilidad Sostenible de una Ciudad

Práctica de la asignatura Programación Orientada a Objetos
Escenario para el curso 2024-2025 - Febrero de 2025 - Versión 1.4

Departamento de Lenguajes y Sistemas Informáticos
Escuela Técnica Superior de Ingeniería Informática - UNED

"Práctica de Programación Orientada a Objetos – Curso 2024-2025"

Memoria Sistema de Gestión de Movilidad Sostenible

Índice:

1. Análisis de la aplicación general.
2. Planteamiento del problema.
    - Conjuntos de actores principales
    - Principales relaciones entre **clases**
    - **Métodos funcionales** implementados en cada clase
3. Diagrama de Clases
4. Explicación del diagrama de Clases
5. Estrategias implementadas y decisiones de diseño
6. Anexo de código fuente de las clases.
    - Explicación métodos con usos prácticos

1. Análisis de la aplicación general

En este punto haré un breve resumen de cómo funciona la estructura que plantea el enunciado de la práctica por encima sin entrar en detalle en las clases o métodos, los cuales se explicarán más adelante.

El objetivo del planteamiento es definir un sistema de movilidad funcional con Personas y Vehículos en el foco,ambos definidos como superclases abstractas para evitar crear instancias directas, y diversas subclases que permitirán modelar roles específicos de comportamiento, usuarios con acceso a los servicios de movilidad, empleados divididos según sus funciones, vehículos diferenciados por tipos…

En conclusión, el sistema gestiona una ciudad con una flota de vehículos, con estaciones base, reservas, alquileres, y registros de estadísticas (batería, ubicación, mantenimiento..).

2. Planteamiento del problema.

Lo primero es establecer un lugar, para ello tendremos la clase ciudad en la cual habrá motos (grandes y pequeñas) y bases, con bicicletas y patinetes, distribuidas por la ciudad.

Para su uso necesitaremos Usuarios que alquilen, reserven vehículos.

Para su mantenimiento y establecer intervenciones críticas se necesitaran empleados para cada rol (administradores, mecánicos y personal de mantenimiento).

Se han definido también las ubicaciones de las bases en las ciudades por coordenadas y la localización de las motos en la ciudad directamente, por coordenadas también.

Por otro lado se han añadido funciones como:

- Según el tipo de usuario (si es o no premium) pueden reservar o no.
- El porcentaje mínimo de batería para alquiler según el tipo de usuario.
- Penalizaciones por agotar la batería.
- Validaciones de los límites de la ciudad y de la posición de los vehículos.

2.1 Conjuntos de actores principales:

- Los tres tipos de trabajadores (Mecanico, Mantenimiento y Administrador), que se encargará de realizar su labores asignadas según su rol.
- Los usuarios, que harán uso del servicio.
- Los vehículos, distinguidos en tres tipos.
- La localización de los vehículos.
- Las gestiones de reservar, alquilar, y poner precios al servicio.

2.2 Principales relaciones entre clases:

Clases	Se relaciona con	Tipo de relación
Persona	Usuario, Empleado	Herencia
Empleado	Mantenimiento, Mecánico, Administrador	Herencia
Vehículo	Bicicleta, Patinete, Moto	Herencia
Moto	MotoPequeña, MotoGrande	Herencia
Vehículo	Tarifa, Coordenadas, Reserva, Base, Usuario	Asociacion / Composición
Avería	Vehículo, Base, Mecánico, Empleado	Asociación
Factura	Avería	Asociación
Las Motos	Coordenadas, Tarifa	Asociación
Usuario	Alquiler, Reserva, Vehiculo, Base, Ciudad	Asociación
Administrador	Usuario, Vehículo, Base, Avería, Ciudad	Asociación
Mecánico	Avería, Factura	Asociación
Alquiler	Vehiculo, Usuario, Coordenadas	Asociación
Mantenimiento	Vehículo	Asociación
Reserva	Vehiculo, Usuario	Asociación Bidireccional
Base	Vehículo, Coordenadas	Contención
Coordenadas	Ciudad, Vehículo, Base	Composición
Bicicleta/Patinete	Base	Agregación
Ciudad	Base, Coordenadas, Vehículo	Agregación / Composición

Explicación del uso de las relaciones más adelante.

2.3 Métodos funcionales implementados en cada clase:

Persona (abstracta)

- mostrarDatos() – Muestra por pantalla la información básica de la persona.

Usuario

- iniciarAlquiler(Vehiculo, Coordenadas, LocalDateTime) – Verifica batería, permiso y saldo. Si es válido, inicia un alquiler.
- finalizarAlquiler(Alquiler, LocalDateTime, Coordenadas) – Termina el alquiler, calcula coste y aplica penalizaciones si es necesario.
- reservarViaje(Vehiculo) – Reserva un vehículo si el usuario es premium y está disponible.
- recargarSaldo(double) – Añade una cantidad al saldo del usuario.
- reportarProblema(Object, String, String) – Crea una avería asociada a un objeto (vehículo o base).
- consultarTodosLosVehiculos( ) – Muestra por consola los vehículos disponibles en bases y sueltos.
- consultarMotoMasCercana( ) – (Simulado) mostrará la moto más cercana a unas coordenadas dadas.

Empleado (abstracta)

- Hereda de Persona. No añade métodos por sí misma.

Administrador

- darAltaUsuario(...) – Crea y añade un nuevo usuario al sistema.
- darBajaUsuario(Usuario) – Elimina un usuario de la lista.
- modificarUsuario(...) – Permite cambiar atributos del usuario (como el email).
- crearBase(...) – Crea una base con nombre, ubicación y espacio, y la añade a la ciudad.
- crearVehiculo(...) – Crea un vehículo del tipo indicado y lo añade a una base o a la flota según su tipo.
- mostrarUsuarios() – Lista los usuarios registrados con sus datos.
- mostrarFlota(...) – Muestra todos los vehículos registrados en la ciudad.
- mostrarBasesDetallado(...) – Muestra las bases existentes, su estado y los vehículos asignados.
- consultarBaterias(...) – Muestra el nivel de batería de todos los vehículos de la flota.
- verAvisos(...) – Lista todas las averías registradas.
- verEstadoBases(...) – Muestra el estado (OK / averiada) de cada base de la ciudad.
- modificarTarifaGlobal(...) – Cambia la tarifa por defecto para un tipo concreto de vehículo.
- mostrarTarifasActuales() – Muestra las tarifas actuales asignadas por tipo de vehículo.

- asignarReparacion(...) – Asocia una avería a un mecánico.
- asignarMantenimiento(...) – Asigna un vehículo al personal de mantenimiento.
- usuariosPromocionables(...) – Devuelve la lista de usuarios que aún no son premium.
- promocionarUsuario(Usuario) – Cambia el estado del usuario a premium.
- modificarDescuentoPremium(...) - Cambia la tarifa Premium para un tipo concreto de vehículo.
- crearEmpleado() - Crea y añade un nuevo Empleado al sistema.
- mostrarResumenEmpleados() - Lista los Empleados registrados con sus datos.

Mantenimiento

- recogerVehiculo(Vehiculo) – Marca un vehículo como en mantenimiento.
- recargarVehiculo(Vehiculo) – Simula la recarga de batería del vehículo.
- entregarVehiculo(Vehiculo) – Vuelve a poner el vehículo como disponible tras revisión.

Mecánico

- asignarAveria(Averia) – Asigna una avería a este mecánico.
- repararAveria(Averia) – Marca la avería como resuelta y libera el objeto.
- generarFactura() – Crea una factura al finalizar una reparación.

Vehículo (abstracta)

- verificarBateria(double) – Devuelve si la batería supera un umbral.
- consumirBateria(int) – Reduce la batería según tarifa y minutos de uso.
- actualizarUbicacion(Coordenadas) – Establece una nueva posición para el vehículo.
- getEstado() – Devuelve el estado actual (disponible, reservado, alquilado…).
- getTarifa() – Devuelve el objeto Tarifa asignado.
- getUbicacion() – Devuelve la posición actual.
- getBateria() – Devuelve el nivel de batería.
- setEstado(String) – Cambia el estado del vehículo.
- setReservaActual(Reserva) – Asigna o elimina una reserva.
- estaReservadoPor(Usuario) – Devuelve si el vehículo está reservado por ese usuario.

Tarifa

- calcularTarifa(boolean, int) – Calcula el coste del viaje según si el usuario es premium.
- getConsumoPorMinuto() – Devuelve el porcentaje de batería que se consume por minuto.
- getDescuentoPremium() – Devuelve el porcentaje de descuento para usuarios premium.
- getTipoVehiculo() – Devuelve el tipo asociado (bicicleta, patinete, etc.).

Reserva

- expiraReserva() – Libera el vehículo si han pasado más de 20 minutos desde la reserva.
- calcularDuracion() – Devuelve cuántos minutos han pasado desde la reserva.

Ciudad

- estaDentroLimites(Coordenadas) – Devuelve si unas coordenadas están dentro del área de la ciudad.
- agregarBase(Base) – Añade una base a la ciudad.
- getBasePorNombre(String) – Busca y devuelve una base por su nombre.
- getBases() – Devuelve todas las bases.
- getFlota() – Devuelve todos los vehículos registrados.

Base

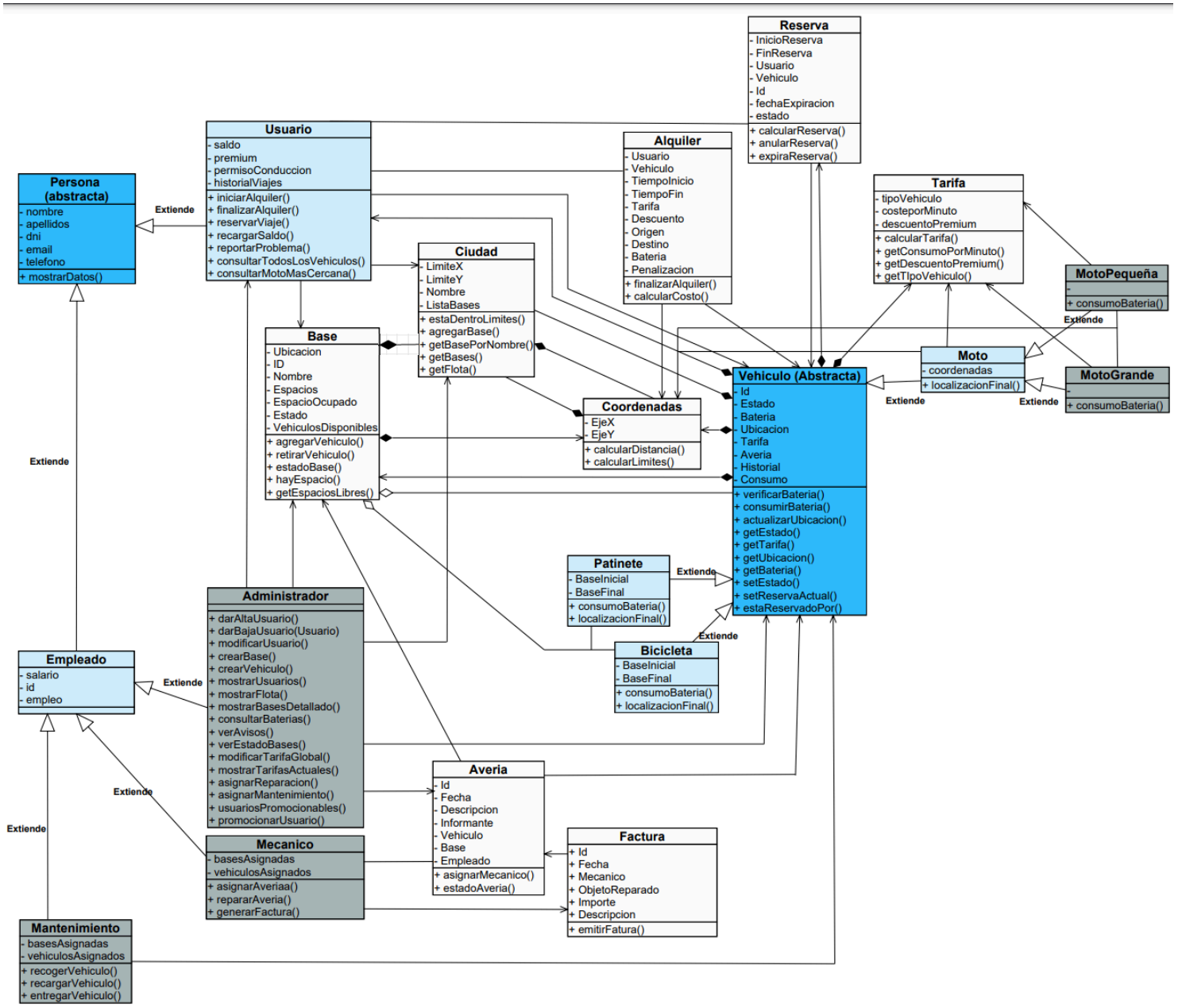
- agregarVehiculo(Vehiculo) – Añade un vehículo si hay espacio disponible.
- retirarVehiculo(Vehiculo) – Elimina un vehículo de la base y lo marca como retirado.
- estadoBase() – Devuelve si la base está operativa o averiada.
- hayEspacio() – Devuelve si hay espacio libre en la base.
- getVehiculos() – Devuelve la lista de vehículos de la base.
- getEspaciosLibres() – Devuelve el número de plazas disponibles.

Alquiler

- finalizarAlquiler(LocalDateTime, Coordenadas) – Marca el alquiler como finalizado.
- calcularCosto() – Calcula el importe total según tiempo y si el usuario es premium.

3. Diagrama de Clases

Para la elaboración de un diagrama de clases antes de empezar a codificar, he empleado una herramienta que incluía elementos para su correcta interpretación, donde son visibles las clases, sus atributos, la privacidad de estos, sus métodos (no he incluido los getters ni los setters porque sería confuso) y las relaciones principales entre los distintos elementos que comentamos anteriormente.



Metodos de Administrador que no se muestran en el diagrama (añadidos mas adelante) modificarDescuentoPremium(...), crearEmpleado(), mostrarResumenEmpleados()

4. Explicación del diagrama de Clases

Relaciones de herencia:

- Persona tiene una relación de herencia con Usuario y Empleado, los cuales heredan sus atributos y no permite crear, por ejemplo una instancia de la clase Persona suelta.

- Empleado funciona de la misma manera con Mecanico, Mantenimiento y Administrador.

- De la misma forma Bicicleta, Patinete y Moto, que a su vez tiene dos herencias más MotoGrande y MotoPequeña.

Relaciones de Asociación:
Las he usado cuando una clase utiliza o se relaciona con otra, pero no la contiene ni la posee, en estos casos serían:

- MotoGrande y MotoPequea con Coordenadas y Tarifa.
- Usuario con Reserva, Alquiler, Vehiculo, Ciudad, Base.
- Administrador con Usuario, Vehiculo, Base, Averia, Ciudad.
- Mecánico con Averia y Factura.
- Reserva con vehiculo y Usuario.
- Alquiler con Vehiculo, Usuario y Coordenadas.
- Factura con Averia.
- Avería con Vehiculo ,Base, Mecánico y Empleado.
- Mantenimiento con Vehículo

Relaciones de composición, elementos que no pueden existir por sí solos por ejemplo, no puede existir un vehículo si no existe una ciudad que albergue este vehículo. En todos los casos serán relaciones 1..* ya que mínimo hará falta una, pero podrán haber más de una.

- Ciudad con Bases, Existirá siempre mínimo 1 ciudad para albergar 0..* bases.
- Ciudad con vehículo y coordenadas igual, no puede existir un vehículo sin Ciudad previa y no pueden existir coordenadas sin una ciudad.
- Coordenadas tiene también este tipo de relación con ciudad y vehículo ya que necesitan existir previamente, no puede haber vehículos sin coordenadas, los patines y bicis están en las bases que necesitan hacer uso de las coordenadas y las motos son vehículos que necesitan también las coordenadas.
- No puede haber reservas si no existen vehículos previamente, relación 1..* Vehículos a 0..* reservas.

Relaciones de agregación:
Vehículos con bases ya que no es necesario que existan bases para que existan vehículos, podrían existir solo Motos que no hacen uso de las bases.
Otro caso seria el de Patinete y Bicicleta con Bases ya que puede existir o no y estos vehículos y la base existir tambien, independientemente de si existen estos o no.

5. Estrategias implementadas y decisiones de diseño

He usado herencias para las clases que se crearán varias veces para evitar duplicación (Moto, Empleado, Persona).

Las tarifas las he unificado en una clase y vinculado a cada tipo de vehículo mediante campos estáticos por defecto.

Validaciones como la batería y el saldo están encapsuladas en el Usuario.

He añadido un control de expiración para las reservas dentro de la propia clase.

Para gestionar la ocupación, se hará desde las bases que gestionan los espacios libres de bicis y patinetes.

El diseño permite extensibilidad: pueden añadirse tipos nuevos sin alterar clases anteriores.

He añadido el atributo de permiso de conducción para poder acceder a las motos.

6. Anexo de código fuente de las clases.

● Añadir y actualizar los datos de la flota de vehículos del sistema, así como de las tarifas aplicadas y los descuentos para usuarios premium.

- Dentro de la clase Administrador:

Añadir flota:

```
/**
 * Crea un vehículo del tipo indicado y lo agrega a la base o ciudad.
 * Habrá que crear antes una ciudad y una base en caso de querer crear Bicis y Patinetes.
 *
 * @param tipo Tipo de vehículo.
 * @param id Identificador del vehículo.
 * @param ubicacion Ubicación deseada.
 * @param base Base (si aplica).
 * @param flota Lista general de vehículos.
 * @param ciudad Ciudad actual.
 */
public void crearVehiculo(String tipo, String id, Coordenadas ubicacion, Base base, ArrayList<Vehiculo> flota, Ciudad ciudad) {
    tipo = tipo.toLowerCase();

    switch (tipo) {
        case "bicicleta":
            if (base != null) {
                Bicicleta bici = new Bicicleta(id);
                base.agregarVehiculo(bici);
                flota.add(bici);
                System.out.println(" Bicicleta " + id + " añadida a base " + base.getNombre());
            } else {
                System.out.println("Las bicicletas deben asignarse a una base.");
            }
            break;

        case "patinete":
            if (base != null) {
                Patinete patinete = new Patinete(id);
                base.agregarVehiculo(patinete);
```

```
                flota.add(patinete);
                System.out.println("Patinete " + id + " añadido a base " + base.getNombre());
            } else {
                System.out.println("Los patinetes deben asignarse a una base.");
            }
            break;

        case "moto pequeña":
            if (ciudad.estaDentroLimites(ubicacion)) {
                MotoPequena mp = new MotoPequena(id, ubicacion);
                flota.add(mp);
                System.out.println("Moto pequeña " + id + " añadida en coordenadas (" + ubicacion.getEjeX() + "," +
ubicacion.getEjeY() + ")");
            } else {
                System.out.println("Ubicación fuera de los límites de la ciudad.");
            }
            break;

        case "moto grande":
            if (ciudad.estaDentroLimites(ubicacion)) {
                MotoGrande mg = new MotoGrande(id, ubicacion);
                flota.add(mg);
                System.out.println("Moto grande " + id + " añadida en coordenadas (" + ubicacion.getEjeX() + "," + ubicacion.getEjeY()
+ ")");
            } else {
                System.out.println("Ubicación fuera de los límites de la ciudad.");
            }
            break;

        default:
            System.out.println("Tipo de vehículo desconocido: " + tipo);
    }
  }
```

Consultar flota:

```
    /**
     * Muestra todos los vehículos registrados en el sistema.
     *
     * @param flota Lista de vehículos.
     */
    public void mostrarFlota(ArrayList<Vehiculo> flota) {
        if (flota.isEmpty()) {
            System.out.println("No hay vehículos registrados en la flota.");
        } else {
            System.out.println("Listado de vehículos:");
            for (Vehiculo v : flota) {
                System.out.println("- ID: " + v.getId() +
                                   " | Tipo: " + v.getClass().getSimpleName() +
                                   " | Estado: " + v.getEstado() +
                                   " | Ubicación: (" + v.getUbicacion().getEjeX() + ", " + v.getUbicacion().getEjeY() + ")");
            }
        }
    }
```

Consulta bases:

```
    /**
     * Muestra el estado de todas las bases del sistema.
     *
     * @param bases Lista de bases.
     */
    public void verEstadoBases(ArrayList<Base> bases) {
        System.out.println("Estado de las bases:");
        for (Base b : bases) {
            System.out.println("Base " + b.getNombre() + " → " + (b.estadoBase() ? "OK" : "Averiada"));
        }
    }
```

Modificador de tarifas:

```
/**
 * Modifica la tarifa por defecto según el tipo de vehículo.
 * @param tipo Tipo de vehículo.
 * @param nuevaTarifa Nueva tarifa.
 */
public void modificarTarifaGlobal(String tipo, Tarifa nuevaTarifa) {
    switch (tipo.toLowerCase()) {
        case "bicicleta":
            Bicicleta.setTarifaPorDefecto(nuevaTarifa);
            break;
        case "patinete":
            Patinete.setTarifaPorDefecto(nuevaTarifa);
            break;
        case "moto pequeña":
            MotoPequena.setTarifaPorDefecto(nuevaTarifa);
            break;
        case "moto grande":
            MotoGrande.setTarifaPorDefecto(nuevaTarifa);
            break;
        default:
            System.out.println("Tipo de vehículo no reconocido: " + tipo);
            return;
        }
    System.out.println("Tarifa modificada para " + tipo);
}
```

Modificador de tarifas Premium:

```
/**
 * Modifica únicamente el descuento premium de una tarifa por tipo de vehículo
 * @param tipo Tipo de vehículo ("bicicleta", "patinete", "moto pequeña", "moto grande").
 * @param nuevoDescuento Nuevo valor de descuento premium (entre 0.0 y 1.0).
 */
public void modificarDescuentoPremium(String tipo, double nuevoDescuento) {
    switch (tipo.toLowerCase()) {
        case "bicicleta":
            Bicicleta.getTarifaPorDefecto().setDescuentoPremium(nuevoDescuento);
            break;
        case "patinete":
            Patinete.getTarifaPorDefecto().setDescuentoPremium(nuevoDescuento);
            break;
        case "moto pequeña":
            MotoPequena.getTarifaPorDefecto().setDescuentoPremium(nuevoDescuento);
            break;
        case "moto grande":
            MotoGrande.getTarifaPorDefecto().setDescuentoPremium(nuevoDescuento);
            break;
        default:
            System.out.println("Tipo de vehículo no reconocido: " + tipo);
            return;
    }
    System.out.println("Descuento premium modificado para " + tipo + ": " + (int)(nuevoDescuento * 100) + "%");
}
```

Consulta de tarifas:

```
/**
 * Muestra las tarifas actuales por tipo de vehículo.
 */
public void mostrarTarifasActuales() {
    System.out.println("Tarifas actuales:");
    System.out.println("Bicicleta: " + Bicicleta.getTarifaPorDefecto());
    System.out.println("Patinete: " + Patinete.getTarifaPorDefecto());
    System.out.println(" Moto Pequeña: " + MotoPequena.getTarifaPorDefecto());
    System.out.println("Moto Grande: " + MotoGrande.getTarifaPorDefecto());
}
```

Crear Bases:

```
    /**
     * Crea una base en la ciudad con nombre, ubicación y espacios.
     *
     * @param nombre Nombre de la base.
     * @param x Coordenada X.
     * @param y Coordenada Y.
     * @param espacios Número de espacios disponibles.
     * @param ciudad Ciudad a la que se agrega la base.
     */
    public void crearBase(String nombre, int x, int y, int espacios, Ciudad ciudad) {
        Coordenadas ubicacion = new Coordenadas(x, y);
        Base nueva = new Base(nombre, ubicacion, espacios);
        ciudad.agregarBase(nueva);
        System.out.println("Base creada: " + nombre + " en (" + x + ", " + y + ") con " + espacios + " espacios.");
    }
```

- Clases de Patines y Bicicletas (funcionan igual, cambia la tarifa asignada por defecto)

```
/**
 * Clase que representa una bicicleta del sistema de movilidad.
 *
 * Hereda de {@link Vehiculo} y utiliza una tarifa por defecto específica para bicicletas.
 * La ubicación inicial se establece posteriormente, normalmente al asignarse a una base.
 *
 * @see Tarifa
 * @see Vehiculo
 *
 * @author [Jesus Lopez Perez]
 * @version [Sistema de Movilidad 2025]
 */
public class Bicicleta extends Vehiculo {
    /** Tarifa por defecto para todas las bicicletas. */
    private static Tarifa tarifaPorDefecto = new Tarifa("bicicleta", 0.15, 0.2, 0.8);

    /**
     * Constructor de la clase Bicicleta.
     *
     * @param id Identificador único de la bicicleta.
     */
    public Bicicleta(String id) {
        super(id, null, tarifaPorDefecto); // La ubicación se asignará posteriormente
    }

    /**
     * Establece una nueva tarifa por defecto para todas las bicicletas.
     *
     * @param nuevaTarifa Nueva tarifa a aplicar como valor por defecto.
     */
    public static void setTarifaPorDefecto(Tarifa nuevaTarifa) {
        tarifaPorDefecto = nuevaTarifa;
    }

    /**
     * Devuelve la tarifa por defecto para bicicletas.
     *
     * @return Tarifa asociada a las bicicletas.
     */
    public static Tarifa getTarifaPorDefecto() {
        return tarifaPorDefecto;
    }
}
```

- Clases de MotoGrande y MotoPequeña (Cambia entre ellas el precio de la tarifa)

```
/**
 * Clase que representa una moto grande del sistema de movilidad.
 *
 * Hereda de {@link Moto} y utiliza una tarifa por defecto específica.
 * Las motos grandes suelen tener mayor coste y mayor consumo que las pequeñas, pero también ofrecen más autonomía y potencia.
 *
 * @author [Jesus Lopez Perez]
 * @version [Sistema de Movilidad 2025]
 */
public class MotoGrande extends Moto {
    /** Tarifa por defecto para todas las motos grandes. */
    private static Tarifa tarifaPorDefecto = new Tarifa("moto grande", 0.30, 0.10, 1.5);

    /**
     * Constructor de la clase MotoGrande.
     *
     * @param id Identificador único de la moto.
     * @param ubicacion Ubicación inicial del vehículo.
     */
    public MotoGrande(String id, Coordenadas ubicacion) {
        super(id, ubicacion, tarifaPorDefecto);
    }

    /**
     * Establece una nueva tarifa por defecto para todas las motos grandes.
     *
     * @param nuevaTarifa Nueva tarifa a aplicar por defecto.
     */
    public static void setTarifaPorDefecto(Tarifa nuevaTarifa) {
        tarifaPorDefecto = nuevaTarifa;
    }

    /**
     * Devuelve la tarifa por defecto actual para motos grandes.
     *
     * @return Tarifa por defecto.
     */
    public static Tarifa getTarifaPorDefecto() {
        return tarifaPorDefecto;
    }
}
```

- **Dentro de la clase Tarifa:**

Constructor de la tarifa con los atributos segun vehiculo, descuento, consumo y coste:

```
    /**
     * Constructor de la clase Tarifa.
     *
     * @param tipoVehiculo Tipo de vehículo (ej. bicicleta, patinete, moto...).
     * @param costeMinuto Coste por minuto de uso en euros.
     * @param descuentoPremium Descuento aplicado a usuarios premium (entre 0 y 1).
     * @param consumoPorMinuto Porcentaje de batería consumido por minuto.
     */
    public Tarifa(String tipoVehiculo, double costeMinuto, double descuentoPremium, double consumoPorMinuto) {
        this.tipoVehiculo = tipoVehiculo;
        this.costeMinuto = costeMinuto;
        this.descuentoPremium = descuentoPremium;
        this.consumoPorMinuto = consumoPorMinuto;
    }
```

Cálculo de coste según duración y si es premium:

```
/**
 * Calcula el precio total del alquiler en función de la duración y si el usuario es premium.
 *
 * @param esPremium {@code true} si el usuario es premium.
 * @param minutos Duración del alquiler en minutos.
 * @return Coste total del alquiler en euros.
 */
public double calcularTarifa(boolean esPremium, int minutos) {
    double tarifa = minutos * costeMinuto;
    if (esPremium) {
        tarifa *= (1 - descuentoPremium);
    }
    return tarifa;
}
```

● Dar de alta a usuarios del sistema con sus datos personales.

Con un administrador:

```
/**
 * Da de alta un nuevo usuario desde cero.
 *
 * @param nombre Nombre del usuario.
 * @param apellidos Apellidos del usuario.
 * @param dni DNI del usuario.
 * @param email Correo electrónico del usuario.
 * @param telefono Teléfono del usuario.
 * @param permisoConduccion Si tiene permiso de conducción.
 */
public void darAltaUsuario(String nombre, String apellidos, String dni, String email, String telefono, boolean permisoConduccion) {
    Usuario nuevo = new Usuario(nombre, apellidos, dni, email, telefono, permisoConduccion);
    usuarios.add(nuevo);
    System.out.println("Usuario dado de alta: " + nombre + " " + apellidos);
}
```

● Realizar el alquiler de vehículos con todas las características asociadas (**gestión de la batería**, **gestión del saldo**, introducción avisos por **fallos mecánicos**, **etc.**).

- **Clase Alquiler:**

```
import java.time.LocalDateTime;
import java.time.Duration;

/**
 * Clase que representa un alquiler de un vehículo realizado por un usuario.
 * El alquiler almacena la información del usuario, el vehículo alquilado, las fechas
 * de inicio y fin, las ubicaciones de origen y destino, así como el importe total.
 * Permite calcular la duración, el coste y el consumo de batería del viaje.
 */
public class Alquiler {
    /** Usuario que realiza el alquiler. */
    private Usuario usuario;

    /** Vehículo alquilado. */
    private Vehiculo vehiculo;

    /** Fecha y hora de inicio del alquiler. */
    private LocalDateTime horaInicio;

    /** Fecha y hora de finalización del alquiler. */
    private LocalDateTime horaFin;
```

```java
/** Ubicación de inicio del alquiler. */
private Coordenadas origen;

/** Ubicación final del vehículo al terminar el alquiler. */
private Coordenadas destino;

/** Importe total del alquiler. */
private double importe;


/**
 * Constructor del alquiler.
 *
 * @param usuario Usuario que realiza el alquiler.
 * @param vehiculo Vehículo alquilado.
 * @param horaInicio Fecha y hora de inicio.
 * @param origen Coordenadas de origen.
 */
public Alquiler(Usuario usuario, Vehiculo vehiculo, LocalDateTime horaInicio, Coordenadas origen) {
    this.usuario = usuario;
    this.vehiculo = vehiculo;
    this.horaInicio = horaInicio;
    this.origen = origen;
    this.importe = 0;
}

/**
 * Finaliza el alquiler, calcula la duración, el coste y actualiza el estado del vehículo.
 * @param destino Coordenadas de destino del vehículo.
 */
public void finalizarAlquiler(Coordenadas destino) {
    this.horaFin = LocalDateTime.now();
    this.destino = destino;

    long duracionMinutos = getDuracionMinutos();
    importe = vehiculo.getTarifa().calcularTarifa(usuario.esPremium(), (int) duracionMinutos);

    vehiculo.actualizarUbicacion(destino);
    vehiculo.consumirBateria((int) duracionMinutos);
    vehiculo.actualizarEstadoTrasUso();
    vehiculo.liberarReserva();
}

/**
 * Devuelve la duración del alquiler en minutos.
 * @return Minutos transcurridos entre el inicio y el fin del alquiler.
 */
public long getDuracionMinutos() {
    return Duration.between(horaInicio, horaFin).toMinutes();
}

/**
 * Calcula el coste del alquiler.
 * @return Importe total del alquiler.
 */
public double calcularCosto() {
    return importe;
}

/**
 * Devuelve el importe total del alquiler.
 * @return Importe pagado por el usuario.
 */
public double getImporte() {
    return importe;
}
```

```
    /**
     * Devuelve el consumo de batería estimado en base a la duración del viaje.
     * @return Consumo de batería total.
     */
    public double getConsumoBateria() {
        return vehiculo.getTarifa().getConsumoPorMinuto() * getDuracionMinutos();
    }

    /**
     * Devuelve el vehículo alquilado.
     * @return Vehículo asociado al alquiler.
     */
    public Vehiculo getVehiculo() {
        return vehiculo;
    }
}
```

- **Dentro de la clase Vehiculo**

Tenemos, por ejemplo, métodos para diferentes actualizaciones:

```
    /**
     * Actualiza la ubicación del vehículo.
     *
     * @param nuevaUbicacion Coordenadas nuevas.
     */
    public void actualizarUbicacion(Coordenadas nuevaUbicacion) {
        this.ubicacion = nuevaUbicacion;
        System.out.println("Ubicación actualizada a (" + nuevaUbicacion.getEjeX() + ", " + nuevaUbicacion.getEjeY() + ")");
    }

    /**
     * Verifica si el nivel de batería supera un umbral mínimo.
     *
     * @param umbral Nivel mínimo exigido.
     * @return {@code true} si la batería es suficiente, {@code false} si no.
     */
    public boolean verificarBateria(double umbral) {
        return bateria >= umbral;
    }

    /**
     * Consume batería en función del tiempo de uso y la tarifa.
     *
     * @param minutos Minutos de uso.
     */
    public void consumirBateria(int minutos) {
        if (tarifa != null) {
            double consumo = minutos * tarifa.getConsumoPorMinuto();
            bateria -= consumo;
            if (bateria < 0) bateria = 0;
        }
    }

    /**
     * Actualiza automáticamente el estado del vehículo tras su uso.
     * Si la batería está baja, pasa a mantenimiento.
     */
    public void actualizarEstadoTrasUso() {
        if (bateria < 10) {
            setEstado("mantenimiento");
        } else {
            setEstado("disponible");
        }
    }
```

- **Dentro de la clase Usuario podemos:**

Reportar averías

```
/**
     * Reporta una avería sobre un objeto del sistema.
     * @param objeto Objeto afectado (vehículo o base).
     * @param descripcion Descripción del problema.
     * @param fecha Fecha del reporte.
     */
    public void reportarProblema(Object objeto, String descripcion, String fecha) {
        Averia nueva = new Averia("AV-" + System.currentTimeMillis(), descripcion, fecha, objeto);
        System.out.println("Avería reportada: " + nueva.getId());
    }
```

Reservar si es premium

```
    /**
     * Reserva un vehículo si el usuario es premium y el vehículo está disponible.
     *
     * @param v Vehículo a reservar.
     */
    public void reservarViaje(Vehiculo v) {
        if (!premium) {
            System.out.println("Solo usuarios premium pueden reservar vehículos.");
            return;
        }

        if (!v.getEstado().equals("disponible")) {
            System.out.println("Este vehículo no está disponible para reserva.");
            return;
        }

        Reserva nueva = new Reserva(this, v);
        v.setReservaActual(nueva);
        v.setEstado("reservado");
        System.out.println("Vehículo reservado por 20 minutos.");
    }
```

Recargar saldo

```
    /**
     * Recarga el saldo del usuario.
     *
     * @param cantidad Cantidad a añadir al saldo.
     */
    public void recargarSaldo(double cantidad) {
        saldo += cantidad;
        System.out.println("Nuevo saldo: " + saldo + " €");
    }
```

Iniciar un Alquiler

```
    /**
     * Inicia un alquiler de un vehículo si se cumplen todas las condiciones necesarias.
     *
     * @param v Vehículo a alquilar.
     * @param origen Ubicación inicial del alquiler.
     * @param horaInicio Hora de inicio (formato texto, no usada directamente).
     */


    public void iniciarAlquiler(Vehiculo v, Coordenadas origen, String horaInicio) {
        if (v instanceof Moto && !permisoConduccion) {
            System.out.println("No puedes alquilar una moto sin permiso de conducción.");
            return;
        }
        if (!v.getEstado().equals("disponible") && !v.estaReservadoPor(this)) {
```

```
            System.out.println("El vehículo no está disponible o está reservado por otro.");
            return;
        }

        double umbral = premium ? 10 : 20;
        if (v.getBateria() < umbral) {
            System.out.println("La batería es insuficiente para alquilar.");
            return;
        }

        double costeEstimado = v.getTarifa().calcularTarifa(premium, 5);
        if (saldo < costeEstimado) {
            System.out.println("Saldo insuficiente para iniciar el alquiler.");
            return;
        }

        Alquiler alquiler = new Alquiler(this, v, LocalDateTime.now(), origen);
        historialViajes.add(alquiler);
        v.setEstado("alquilado");
        v.setReservaActual(null);
        System.out.println("Alquiler iniciado con éxito.");
    }
```

● Gestionar toda la funcionalidad relativa a los usuarios premium: consulta de **potenciales usuarios premium** por parte del administrador, **asignación del estatus** de premium a usuarios, **gestión de los descuentos y de la posibilidad de realizar reservas**. Gestión de la posibilidad por parte de un usuario premium de alquilar vehículos con un nivel de batería entre un 10% y un 20%.

La gestión de descuentos las realizo al modificar la tarifa y la posibilidad de hacer reservas, según el enunciado, es solo para usuarios Premium, si el usuario es premium puede reservar.
Según el nivel de batería que vimos anteriormente, si está en el umbral para según qué tipo de usuario, el vehículo está o no disponible para su reserva u alquiler.

- **Dentro de la clase Administrador:**

Consulta de usuarios potenciales:

```
    /**
     * Devuelve usuarios no premium que pueden ser promocionados.
     * @param lista Lista de usuarios.
     * @return Lista de usuarios promocionables.
     */
    public ArrayList<Usuario> usuariosPromocionables(ArrayList<Usuario> lista) {
        ArrayList<Usuario> candidatos = new ArrayList<>();
        for (Usuario u : lista) {
            if (!u.esPremium()) {
                candidatos.add(u);
            }
        }
        return candidatos;
    }
```

Promocion de usuarios:

```
    /**
     * Promociona un usuario a premium.
     * @param u Usuario a promocionar.
     */
    public void promocionarUsuario(Usuario u) {
        if (!u.esPremium()) {
            u.setPremium(true);
            System.out.println("Usuario " + u.getNombre() + " ahora es Premium.");
        }
    }
```

● Asignar vehículos con baja batería o con fallos mecánicos a encargados de mantenimiento para su retirada, y gestionar.

Como vimos anteriormente en la clase Vehiculo tenemos el metodo actualizarEstadoTrasUso()
Para establecer el estado mantenimiento si la bateria es inferior a 10%, cuando esto pasa el Empleado Mantenimiento, tiene una clase para asignarse vehículos que estén en el estado mantenimiento.

```
/**
 * Asigna un vehículo al empleado de mantenimiento.
 * @param v Vehículo que se desea asignar.
 */
public void asignarVehiculo(Vehiculo v) {
    vehiculosAsignados.add(v);
    System.out.println("Vehículo asignado a mantenimiento.");
}

/**
 * Muestra por consola los vehículos actualmente asignados al empleado.
 */
public void verVehiculosAsignados() {
    for (Vehiculo v : vehiculosAsignados) {
        System.out.println("Vehículo ID: " + v.id + " - Estado: " + v.estado);
    }
}

/**
 * Cambia el estado del vehículo a "en mantenimiento".
 * @param v Vehículo que se desea recoger para mantenimiento.
 */
public void recogerVehiculo(Vehiculo v) {
    if (vehiculosAsignados.contains(v)) {
        v.estado = "en mantenimiento";
        System.out.println("Vehículo recogido para mantenimiento.");
    }
}

/**
 * Recarga completamente la batería del vehículo.
 * @param v Vehículo cuya batería se va a recargar.
 */
public void recargarBateria(Vehiculo v) {
    if (vehiculosAsignados.contains(v)) {
        v.bateria = 100;
        System.out.println("Batería recargada.");
    }
}

/**
 * Devuelve el vehículo a su ubicación final tras el mantenimiento.
 * @param v Vehículo que se desea devolver.
 * @param destino Coordenadas donde se dejará el vehículo.
 */
public void devolverVehiculo(Vehiculo v, Coordenadas destino) {
    if (vehiculosAsignados.contains(v)) {
        v.ubicacion = destino;
        v.estado = "disponible";
        System.out.println("Vehículo devuelto a ubicación: (" + destino.getEjeX() + ", " + destino.getEjeY() + ")");
    }
}
```

● Permitir que cada encargado de mantenimiento y mecánico vea los vehículos que tiene asignados y realice sus funciones sobre los mismos, dejando constancia, en el caso de los mecánicos, del importe de las reparaciones realizadas.

Dentro de la clase mecanico podemos encontrar los siguientes métodos para dichas cuestiones relacionadas con sus funciones:

```
/**
 * Asigna una avería al mecánico y marca su estado como "en reparación".
 *
 * @param a Avería que se va a asignar.
 */
public void asignarAveria(Averia a) {
    averiasAsignadas.add(a);
    a.asignarEmpleado(this); // marca estado como "en reparación"
}

/**
 * Muestra por consola todas las averías asignadas al mecánico junto con su estado.
 */
public void consultarAverias() {
    for (Averia a : averiasAsignadas) {
        System.out.println("Avería ID: " + a.getId() + " - Estado: " + a.getEstado());
    }
}

/**
 * Marca una avería como resuelta si está asignada al mecánico.
 *
 * @param a Avería que se desea reparar.
 */
public void repararAveria(Averia a) {
    if (averiasAsignadas.contains(a)) {
        a.marcarResuelta();
        System.out.println("Avería " + a.getId() + " reparada.");
    }
}

/**
 * Genera una factura tras la reparación de una avería.
 *
 * @param id Identificador de la factura.
 * @param fecha Fecha de emisión de la factura.
 * @param objetoReparado Objeto que fue reparado.
 * @param importe Importe total de la reparación.
 * @return Objeto {@link Factura} generado.
 */
public Factura generarFactura(String id, String fecha, Object objetoReparado, double importe) {
    return new Factura(id, fecha, objetoReparado, importe);
}
```

● Realizar búsquedas sencillas sobre los usuarios y empleados del sistema.

Realizadas desde el Empleado Administrador, consultar los usuarios como vimos más arriba, los cuales se almacenan en la clase Administrador en el array:

```
private ArrayList<Usuario> usuarios = new ArrayList<>();


public void mostrarUsuarios() …
```

Para consultar y registrar empleados:

```
    /**
     * Crea y registra un nuevo empleado del tipo indicado.
     *
     * @param tipo Tipo de empleado ("mecanico", "mantenimiento", "administrador").
     * @param nombre Nombre del empleado.
     * @param apellidos Apellidos del empleado.
     * @param dni DNI del empleado.
     * @param email Correo electrónico.
     * @param telefono Teléfono de contacto.
     * @param id Identificador único.
     * @param salario Salario mensual.
     */
    public void crearEmpleado(String tipo, String nombre, String apellidos, String dni, String email, String telefono, int id, double
salario) {
        Empleado nuevo = null;

        switch (tipo.toLowerCase()) {
            case "mecanico":
                nuevo = new Mecanico(nombre, apellidos, dni, email, telefono, id, salario);
                break;
            case "mantenimiento":
                nuevo = new Mantenimiento(nombre, apellidos, dni, email, telefono, id, salario);
                break;
            case "administrador":
                nuevo = new Administrador(nombre, apellidos, dni, email, telefono, id, salario);
                break;
            default:
                System.out.println("Tipo de empleado no reconocido: " + tipo);
                return;
        }

        empleados.add(nuevo);
        System.out.println(tipo.substring(0, 1).toUpperCase() + tipo.substring(1).toLowerCase() +
                           " registrado: " + nuevo.getNombre() + " " + nuevo.getApellidos());
    }


    /**
     * Muestra un resumen con el total de empleados registrados clasificados por tipo.
     */
    public void mostrarResumenEmpleados() {
        int totalMecanicos = 0;
        int totalMantenimiento = 0;
        int totalAdministradores = 0;

        for (Empleado e : empleados) {
            if (e instanceof Mecanico) totalMecanicos++;
            else if (e instanceof Mantenimiento) totalMantenimiento++;
            else if (e instanceof Administrador) totalAdministradores++;
        }

        int total = empleados.size();

        System.out.println("Resumen de empleados:");
        System.out.println("Total: " + total);
        System.out.println("Mecanicos: " + totalMecanicos);
        System.out.println("Mantenimiento: " + totalMantenimiento);
        System.out.println("Administradores: " + totalAdministradores);
    }
```

- Realizar consultas y actualizaciones del stock de vehículos del sistema.

Desde la clase Administrador:

```
/**
 * Muestra el nivel de batería de cada vehículo.
 *
 * @param flota Lista de vehículos.
 */
public void consultarBaterias(ArrayList<Vehiculo> flota) {
    System.out.println("Niveles de batería de la flota:");
    for (Vehiculo v : flota) {
        System.out.println(" - " + v.getId() + ": " + v.getBateria() + "%");
    }
}

/**
 * Muestra todos los vehículos registrados en el sistema.
 *
 * @param flota Lista de vehículos.
 */
public void mostrarFlota(ArrayList<Vehiculo> flota) {
    if (flota.isEmpty()) {
        System.out.println("No hay vehículos registrados en la flota.");
    } else {
        System.out.println("Listado de vehículos:");
        for (Vehiculo v : flota) {
            System.out.println("- ID: " + v.getId() +
                               " | Tipo: " + v.getClass().getSimpleName() +
                               " | Estado: " + v.getEstado() +
                               " | Ubicación: (" + v.getUbicacion().getEjeX() + ", " + v.getUbicacion().getEjeY() + ")");
        }
    }
}

/**
 * Muestra información detallada de cada base y sus vehículos.
 *
 * @param bases Lista de bases del sistema.
 */
public void mostrarBasesDetallado(ArrayList<Base> bases) {
    System.out.println("Información de todas las bases:");
    for (Base b : bases) {
        System.out.println("Base: " + b.getNombre() +
                           " | Ubicación: (" + b.getUbicacion().getEjeX() + "," + b.getUbicacion().getEjeY() + ")" +
                           " | Estado: " + (b.estadoBase() ? "OK" : "Averiada") +
                           " | Ocupación: " + b.getVehiculos().size() + "/" + b.getEspacios());

        for (Vehiculo v : b.getVehiculos()) {
            System.out.println(" Vehículo: " + v.getId() + " | Tipo: " + v.getClass().getSimpleName() + " | Estado: " + v.getEstado());
        }
    }
}
```

● Permitir la búsqueda de los vehículos más cercanos: que el usuario pueda consultar cuáles son las bases de bicicletas o patinetes, o las motos individuales, más cercanas a su posición.

Dentro de la clase Usuario, para mostrar los vehículos disponibles y su posición

```
/**
 * Muestra todos los vehículos disponibles, tanto en bases como sueltos.
 *
 * @param bases Lista de bases.
 * @param sueltos Lista de vehículos fuera de bases.
 */
public void consultarTodosLosVehiculos(ArrayList<Base> bases, ArrayList<Vehiculo> sueltos) {
    System.out.println("Vehículos en bases:");
    for (Base b : bases) {
        System.out.println("Base: " + b.getNombre());
        for (Vehiculo v : b.getVehiculos()) {
            System.out.println(" - " + v.getId() + " (" + v.getClass().getSimpleName() + ")");
        }
    }

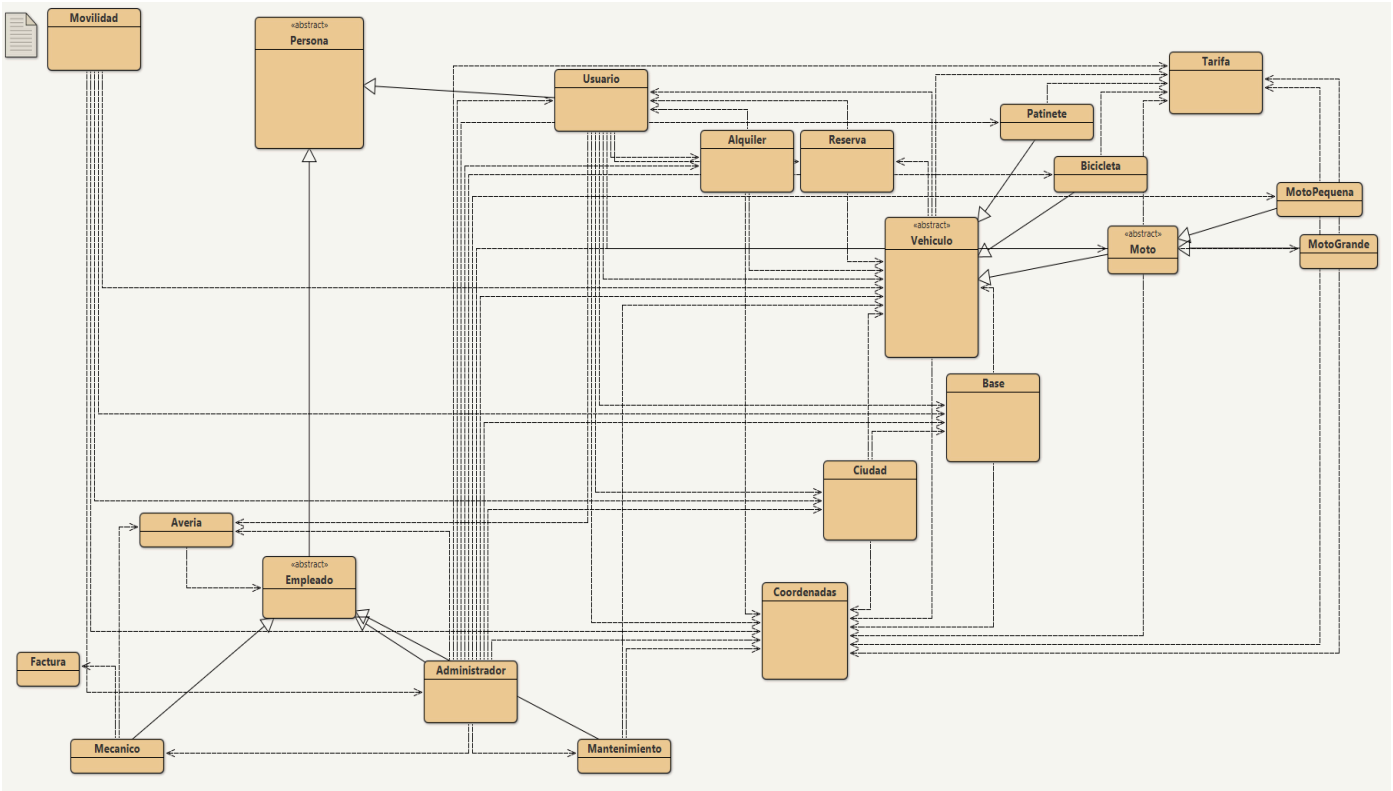
    System.out.println("Vehículos fuera de bases:");
    for (Vehiculo v : sueltos) {
        System.out.println(" - " + v.getId() + " (" + v.getClass().getSimpleName() + ")");
    }
}
```

Tengo tambien la clase consultarMotoMasCercana aun sin desarrollar:

```
/**
 * Método de simulación para buscar la moto más cercana.
 *
 * @param ciudad Ciudad en la que se realiza la consulta.
 * @param posicion Coordenadas del usuario.
 */
public void consultarMotoMasCercana(Ciudad ciudad, Coordenadas posicion) {
    System.out.println("Consulta de moto más cercana no implementada.");
}
```

6.1 Explicación métodos con usos prácticos

Diagrama resultante en BlueJ



Batería de pruebas desde la clase Movilidad:

```
import java.util.ArrayList;
/**
 * Clase principal
 * Ejecuta un escenario de pruebas
 */

public class Movilidad {
    // Atributos estáticos para poder usarlos desde la interfaz si se desea
    public static Ciudad ciudad;
    public static Administrador admin;

    public static void main(String[] args) {
        // Crear ciudad
        ciudad = new Ciudad("CiudadX", 10, 10); // 10x10 de coordenadas
        ArrayList<Vehiculo> flota = ciudad.getFlota();
        ArrayList<Base> bases = ciudad.getBases();

        // Crear administrador
        admin = new Administrador("Jesus", "Lopez", "23065805w", "jesuslopez_93@hotmail.com",
"60235234453", 1, 3000.0);

        // Crear bases
        admin.crearBase("Norte", 2, 3, 3, ciudad);
        admin.crearBase("Sur", 7, 8, 2, ciudad);

       // Obtener referencias a las bases
        Base baseNorte = ciudad.getBasePorNombre("Norte");
        Base baseSur = ciudad.getBasePorNombre("Sur");

        // Crear bicicletas y patinetes en las bases
        admin.crearVehiculo("bicicleta", "BIC-001", null, baseNorte, flota, ciudad);
        admin.crearVehiculo("bicicleta", "BIC-002", null, baseSur, flota, ciudad);
        admin.crearVehiculo("patinete", "PAT-001", null, baseNorte, flota, ciudad);


        // Crear motos fuera de base
        admin.crearVehiculo("moto pequeña", "MP-001", new Coordenadas(5, 5), null, flota, ciudad);
        admin.crearVehiculo("moto grande", "MG-001", new Coordenadas(9, 9), null, flota, ciudad);

        // Crear usuarios (uno con permiso de conducción, otro sin)
        admin.darAltaUsuario("Lucía", "Gómez", "12345678L", "lucia@mail.com", "611111111", true);
        admin.darAltaUsuario("Carlos", "Ruiz", "87654321C", "carlos@mail.com", "622222222", false);

        // Mostrar resumen general por consola
        System.out.println("\n USUARIOS:");
        admin.mostrarUsuarios();

        System.out.println("\n BASES:");
        admin.mostrarBasesDetallado(bases);

        System.out.println("\n FLOTA COMPLETA:");
        admin.mostrarFlota(flota);

        // Confirmación de ejecución correcta
        System.out.println("\n Test básico del sistema\n");
    }
}
```

Salida:

```
Opciones
 Bicicleta BIC-001 añadida a base Norte
Ubicación actualizada a (7, 8)
 Bicicleta BIC-002 añadida a base Sur
Ubicación actualizada a (2, 3)
Patinete PAT-001 añadido a base Norte
Moto pequeña MP-001 añadida en coordenadas (5,5)
Moto grande MG-001 añadida en coordenadas (9,9)
Usuario dado de alta: Lucía Gómez
Usuario dado de alta: Carlos Ruiz

USUARIOS:
Lista de usuarios registrados:
Lucía Gómez (12345678L)
Carlos Ruiz (87654321C)

BASES:
Información de todas las bases:
Base: Norte | Ubicación: (2,3) | Estado: OK | Ocupación: 2/3
Vehículo: BIC-001 | Tipo: Bicicleta | Estado: disponible
Vehículo: PAT-001 | Tipo: Patinete | Estado: disponible
Base: Sur | Ubicación: (7,8) | Estado: OK | Ocupación: 1/2
Vehículo: BIC-002 | Tipo: Bicicleta | Estado: disponible

FLOTA COMPLETA:
Listado de vehículos:
- ID: BIC-001 | Tipo: Bicicleta | Estado: disponible | Ubicación: (2, 3)
- ID: BIC-002 | Tipo: Bicicleta | Estado: disponible | Ubicación: (7, 8)
- ID: PAT-001 | Tipo: Patinete | Estado: disponible | Ubicación: (2, 3)
- ID: MP-001 | Tipo: MotoPequena | Estado: disponible | Ubicación: (5, 5)
- ID: MG-001 | Tipo: MotoGrande | Estado: disponible | Ubicación: (9, 9)

 Test basico del sistema
```

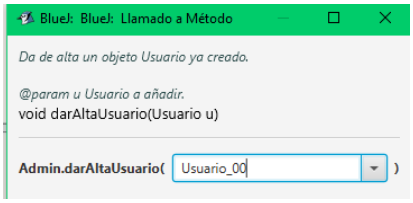
Ejemplo creando Usuarios y mostrandolos:

Secuencia de la prueba:

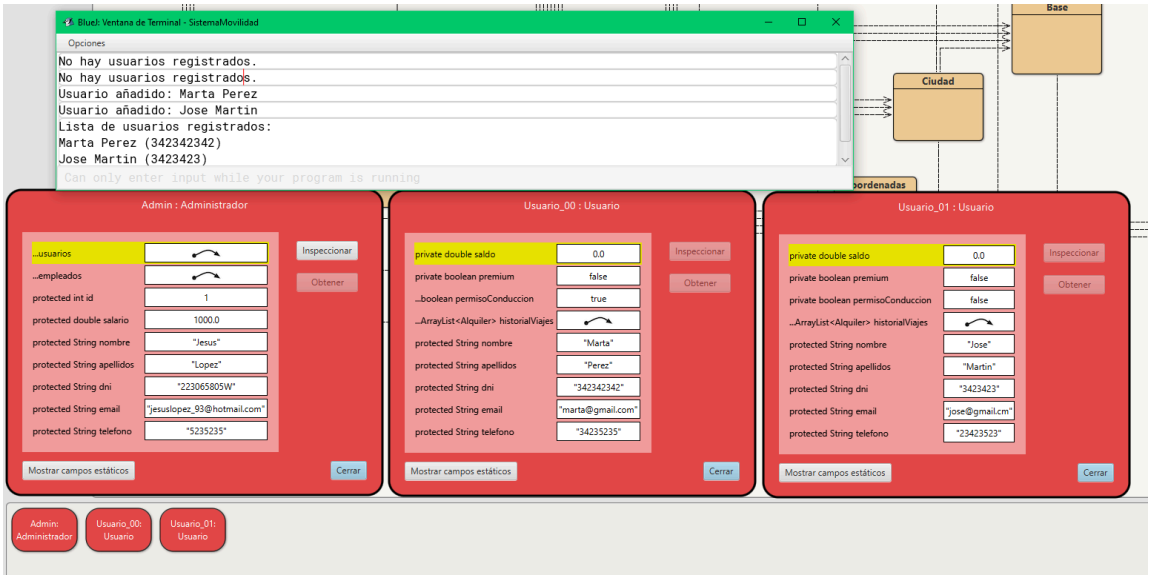
1. Creamos con la interfaz de BlueJ los Objetos, Administrador, Usuario_00 y Usuario_01.
2.
3. Dentro del objeto administrador:

   Ejecutamos el método mostrarUsuarios(), nos sale el mensaje No hay usuarios registrados, falta darlos de alta.

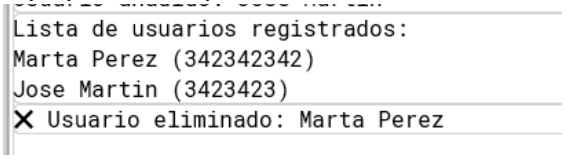
   Ejecutamos el método dar AltaUsuario (Usuario U) para añadirlo a nuestra lista de Usuarios.



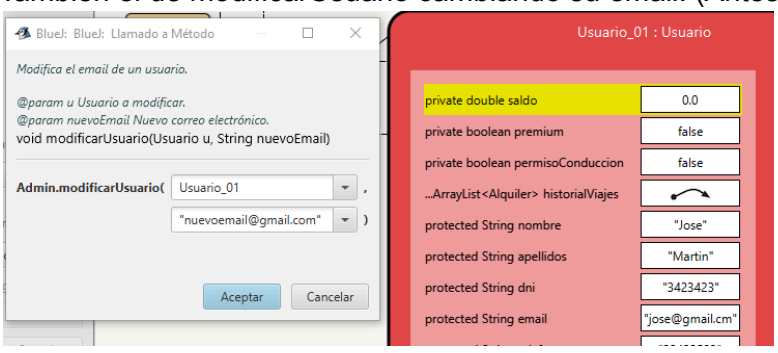
   Ahora ejecutando de nuevo mostrarUsuarios nos saldrán los Usuarios registrados.

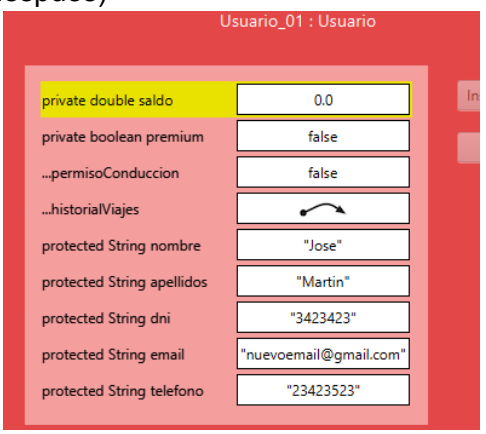


Aqui podremos probar metodos como dar de Baja a usuario tambíen con el resultado:



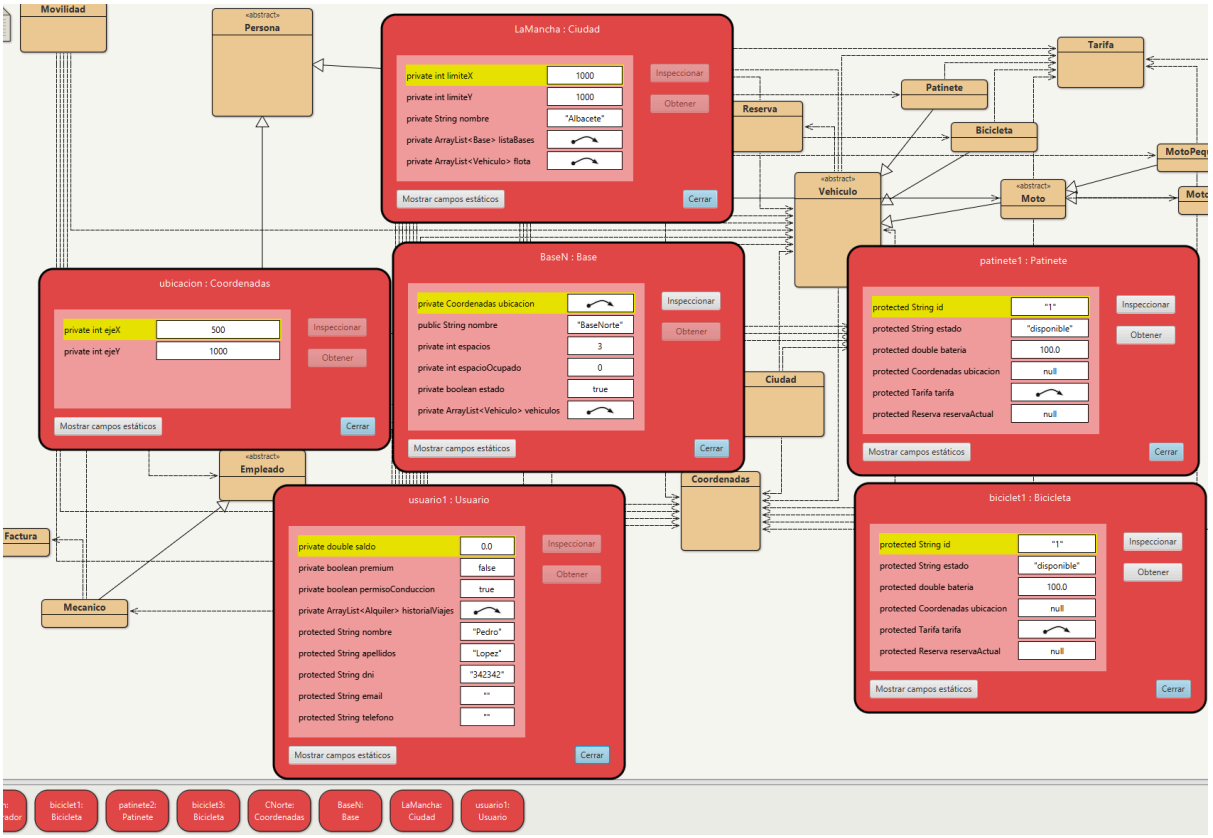
También el de modificarUsuario cambiando su email: (Antes y despues)



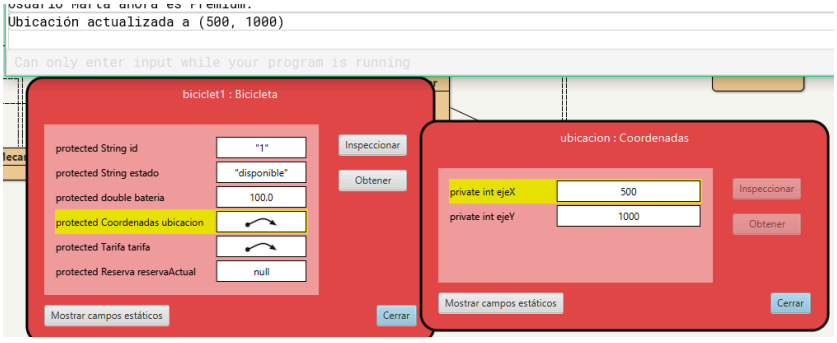


Creando vehículos, bases y ciudades:

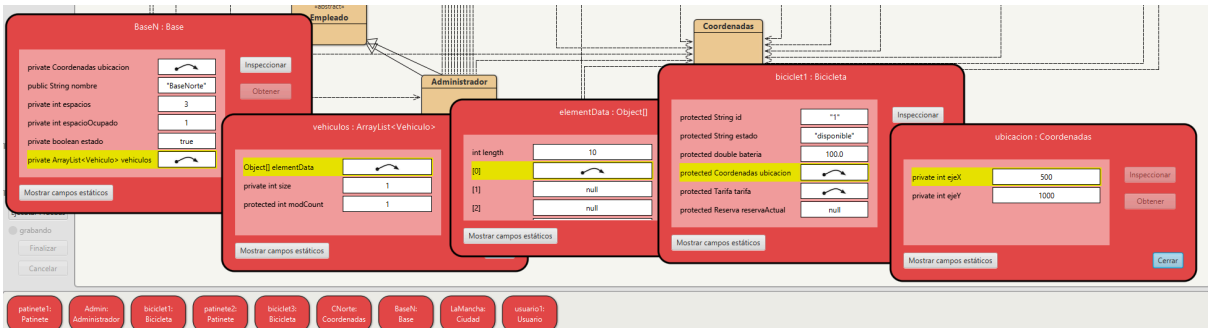
Creamos los vehículos con la interfaz gráfica de BlueJ.



Desde nuestra Base creada agregamos uno de los vehículos creados (biciclet1 en la BaseN)

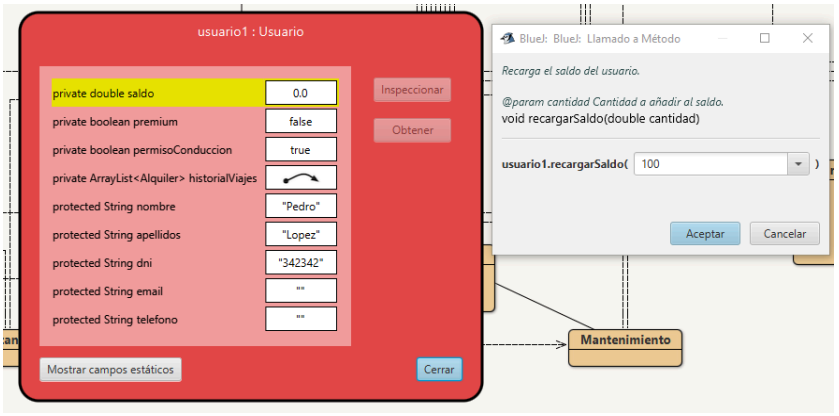


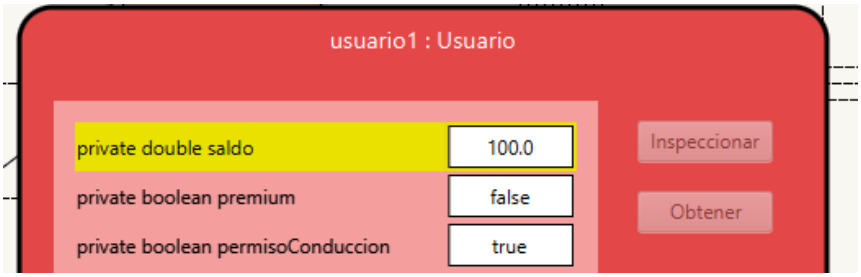
Verificamos que se ha añadido y se han actualizado los espacios de esa base:



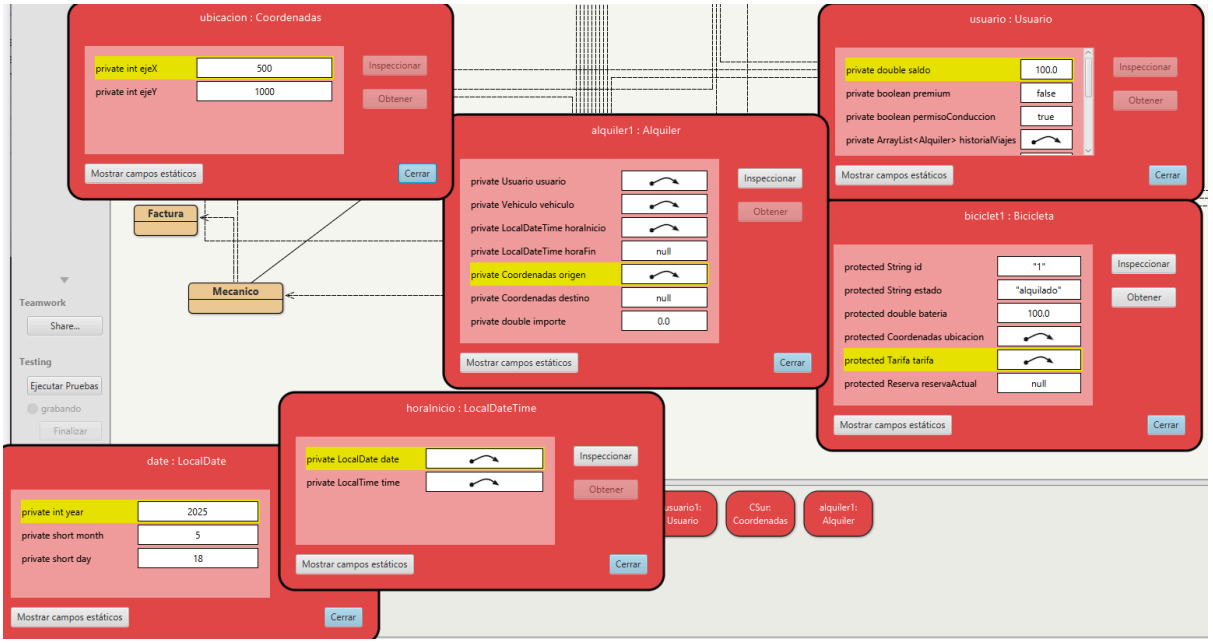
Gestiones y consultas con Usuarios:

Recargar saldo:

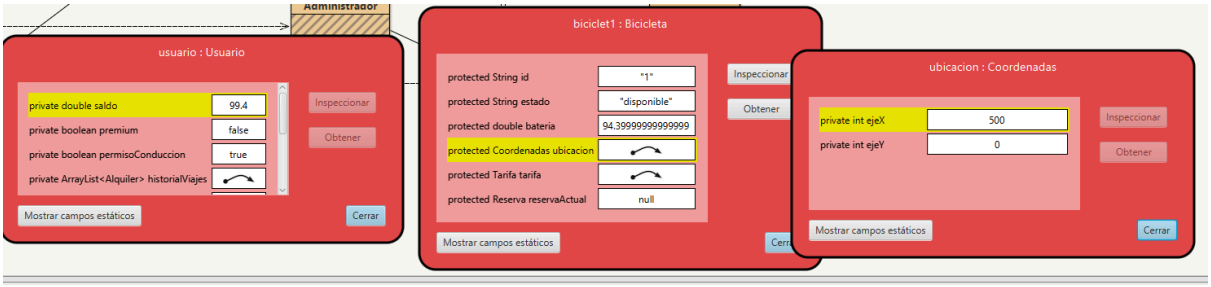




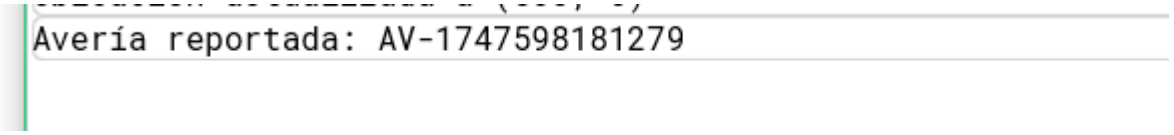
Alquilamos el vehículo biciclet1 desde la clase Alquiler al comprobar vemos que el usuario ha alquilado el vehículo y el estado de la bicicleta cambió.



Finalizamos el alquiler desde el objeto Usuario y alquiler1, revisamos los campos del Usuario y la Bicicleta y vemos que se han actualizado correctamente.

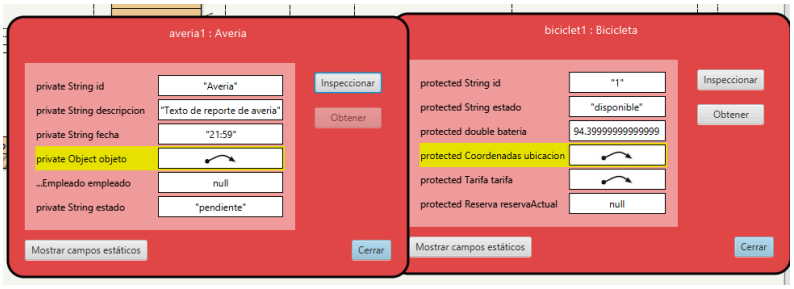


Probamos que se pueden crear Averias desde la clase Usuario mediante la funcion reportarAverias.



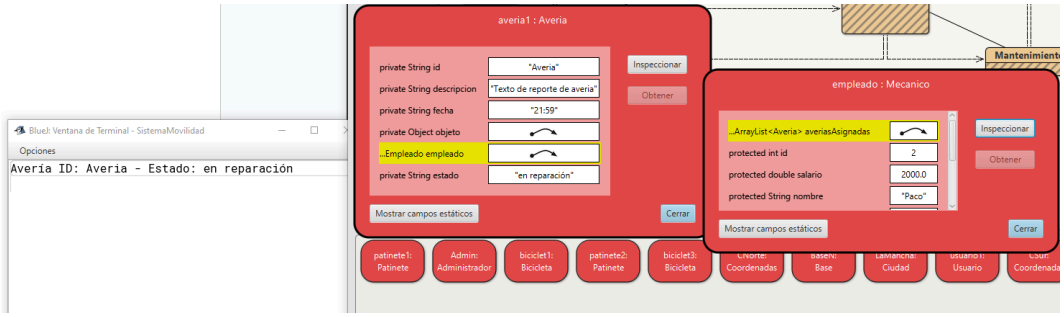
Creamos una nueva Averia de forma grafica para interactuar con la clase Mecanico:

La creamos con referencia a la clase biciclet1, vemos que aun no tiene empleado asignado.



Tras asignarse vemos que se actualizan correctamente los campos:

También podemos consultar las averias asignadas una vez



Podemos también generar una factura de dicho vehículo: con el método de generarFactura.

Se adjunta el proyecto en BlueJ junto con la doc SistemaMovilidad/doc/package-summary.html